

Python Exercise — Dijkstra’s Shortest Path Algorithm (Answers)

Jiwon Kim, The University of Queensland

2026

Contents

Python Exercise — Dijkstra’s Shortest Path Algorithm (Answers)	2
Starting Point	2
Task 1 — Add a Cost Column	3
Task 2 — Initialise the Algorithm	4
Task 3 — Dijkstra’s Main Loop	4
Task 4 — Reconstruct the Shortest Paths	5
Reflection Questions	6

Python Exercise — Dijkstra's Shortest Path Algorithm (Answers)

Starting Point

```
import pandas as pd
import numpy as np

# Tells NumPy to use the legacy print format, which prints plain numbers instead of
# verbose types (e.g. '3.0' instead of 'np.float64(3.0)')
np.set_printoptions(legacy='1.25')

# node_data has three columns: the node ID, and x/y coordinates for plotting.
node_data = {
    'node': [1, 2, 3, 4],
    'x':    [0, 1, 0, 1],
    'y':    [1, 1, 0, 0],
}
# Convert the node_data dictionary into a DataFrame.
df_nodes = pd.DataFrame(node_data)
# Set the node ID as the index for easy lookup by node ID later.
df_nodes.set_index('node', inplace=True, drop=False)

# link_data has three columns: the upstream node (init_node), the downstream
# node (term_node), and the link length in km.
link_data = {
    'init_node': [1, 1, 3, 3, 4],
    'term_node': [2, 3, 2, 4, 2],
    'length':    [1.0, 1.2, 1.5, 1.1, 1.0],
}
# Convert the link_data dictionary into a DataFrame.
df_links = pd.DataFrame(link_data)
# Set a MultiIndex on df_links for easy lookup by (init_node, term_node).
df_links.set_index(['init_node', 'term_node'], inplace=True, drop=False)

# Print the link and node DataFrames to check they look correct.
print("--- Network Data ---")
print("df_nodes:")
print(df_nodes)
print("\ndf_links:") # \n adds a blank line before the next print for readability
print(df_links)

# Construct the forward star dictionary to store the downstream nodes for each node.
forward_star = {}

# Loop through all nodes in df_nodes to initialise forward_star with empty lists.
for row in df_nodes.itertuples(index=False):
    forward_star[row.node] = []

# Loop through all links in df_links to populate the forward_star dictionary.
for row in df_links.itertuples(index=False):
```

```

        forward_star[row.init_node].append(row.term_node)

print("\nforward_star:", forward_star)

```

Output:

```

--- Network Data ---
df_nodes:
      node  x  y
node
1         1  0  1
2         2  1  1
3         3  0  0
4         4  1  0

df_links:
      init_node  term_node  length
init_node term_node
1         2         1         2     1.0
         3         1         3     1.2
3         2         3         2     1.5
         4         3         4     1.1
4         2         4         2     1.0

forward_star: {1: [2, 3], 2: [], 3: [2, 4], 4: [2]}

```

Task 1 — Add a Cost Column

Approach 1 — row-by-row with `itertuples()` (consistent with the `week4.py` pattern):

```

df_links['cost'] = 0.0

for row in df_links.itertuples(index=True):
    df_links.at[row.Index, 'cost'] = row.length * 2

print("\n--- Link Data with Costs ---")
print(df_links)

```

Approach 2 — vectorised column assignment (shorter):

```

df_links['cost'] = df_links['length'] * 2

print("\n--- Link Data with Costs ---")
print(df_links)

```

Both produce the same result. Approach 2 is more concise; Approach 1 is useful when the cost formula is more complex or depends on multiple conditions per row.

Output:

```

--- Link Data with Costs ---

```

		init_node	term_node	length	cost
init_node	term_node				
1	2	1	2	1.0	2.0
	3	1	3	1.2	2.4
3	2	3	2	1.5	3.0
	4	3	4	1.1	2.2
4	2	4	2	1.0	2.0

Task 2 — Initialise the Algorithm

```
# set the origin node ID
orig_node = 1

L = {} # label: shortest path cost from origin to each node
q = {} # backnode: preceding node on the current shortest path
Q = [] # unvisited nodes

# Loop through all nodes in df_nodes to initialise L, q, and Q.
for row in df_nodes.itertuples(index=False):
    L[row.node] = np.inf
    q[row.node] = -1
    Q.append(row.node)

# Set the label (shortest path cost) of the origin node to 0.
L[orig_node] = 0.0

print("\n--- Algorithm Initialisation ---")
print("orig_node:", orig_node)
print("L:", L)
print("q:", q)
print("Q:", Q)
```

Output:

```
--- Algorithm Initialisation ---
orig_node: 1
L: {1: 0.0, 2: inf, 3: inf, 4: inf}
q: {1: -1, 2: -1, 3: -1, 4: -1}
Q: [1, 2, 3, 4]
```

Task 3 — Dijkstra's Main Loop

```
while len(Q) > 0:

    # Step 1: find the unvisited node with the smallest label
    i = -1
    minL = np.inf
```

```

for u in Q:
    if L[u] <= minL:
        minL = L[u]
        i = u

# Step 2: remove i from Q
Q.remove(i)

# Step 3: update labels of downstream nodes still in Q
for j in forward_star[i]:
    if j in Q:
        t_ij = df_links.at[(i, j), 'cost']
        newL = L[i] + t_ij
        if newL < L[j]:
            L[j] = newL
            q[j] = i

print("\n--- Algorithm Results ---")
print("L:", L)
print("q:", q)

```

Output:

```

--- Algorithm Results ---
L: {1: 0.0, 2: 2.0, 3: 2.4, 4: 4.6}
q: {1: -1, 2: 1, 3: 1, 4: 3}

```

Task 4 — Reconstruct the Shortest Paths

```

# Create a dictionary to store the shortest path (node sequence) for each
# origin-destination pair (origin node, destination node)
odpath = {}

# Given the orig_node, loop through all nodes in df_nodes to reconstruct
# the shortest path to each destination node.
for row in df_nodes.iterrows(index=False):
    dest_node = row.node

    # Skip the origin node and any unreachable nodes (those with label = infinity).
    if dest_node == orig_node or L[dest_node] == np.inf:
        continue

    # Start from the list containing the destination node, and keep inserting
    # the backnode at the front of the list until we reach the origin node
    # (which has backnode = -1).
    path = [dest_node]
    current = dest_node

    while q[current] != -1:

```

```

    path.insert(0, q[current])
    current = q[current]

# Store the path in the odpath dictionary with key (orig_node, dest_node).
odpath[(orig_node, dest_node)] = path

print("\n--- Shortest Paths ---")
for od, path in odpath.items():
    # od is a tuple of (orig_node, dest_node), where od[0] is orig_node and
    # od[1] is dest_node.
    print(f"{od}: {path} (cost: {L[od[1]]})")

```

Output:

```

--- Shortest Paths ---
(1, 2): [1, 2] (cost: 2.0)
(1, 3): [1, 3] (cost: 2.4)
(1, 4): [1, 3, 4] (cost: 4.6)

```

Reflection Questions

1. **Change `orig_node = 3`.** Only one line needs to change. Re-running Tasks 2–4 gives:

```

--- Algorithm Initialisation ---
orig_node: 3
L: {1: inf, 2: inf, 3: 0.0, 4: inf}
q: {1: -1, 2: -1, 3: -1, 4: -1}
Q: [1, 2, 3, 4]

--- Algorithm Results ---
L: {1: inf, 2: 3.0, 3: 0.0, 4: 2.2}
q: {1: -1, 2: 3, 3: -1, 4: 3}

--- Shortest Paths ---
(3, 2): [3, 2] (cost: 3.0)
(3, 4): [3, 4] (cost: 2.2)

```

Node 1 is unreachable from node 3 (no outgoing path exists from 3 to 1), so $L[1] = \infty$ and it does not appear in `odpath`.

2. **Change `orig_node = 2`.** Node 2 has no outgoing links (`forward_star[2] = []`), so no node can be reached from it. After the algorithm runs, all labels remain ∞ and `odpath` is empty — the loop `for od, path in odpath.items()` produces no output, leaving only the bare header.

To inform the user, add a guard after building `odpath`. Note that `not odpath` is equivalent to `len(odpath) == 0` — an empty dict is falsy in Python, so this is the idiomatic way to check for emptiness:

```

print("\n--- Shortest Paths ---")
if not odpath:
    print(f"No paths found from origin node {orig_node}.")
else:

```

```
for od, path in odpath.items():  
    print(f"{od}: {path} (cost: {L[od[1]])")
```

This prints:

--- Shortest Paths ---

No paths found from origin node 2.